



Chapter 5-Requirements Verification & Validation



Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

- **Why the Requirements V&V? Recalling Back:** Key terms about RE Elicitation and Specification
 - **Elicitation** is **problem oriented** and **NOT solution oriented**.
 - Is about discovering the boundaries of the right system to build.
 - Is about **collaborative** work between **Stakeholders** and **Requirements Engineers**.
 - Is not spontaneous.
 - Is about making explicit what is implicit.
 - Different techniques (observations, interviews, brainstorming, modeling...)
 - **Specification** is about writing clear, precise, unambiguous, atomic, complete, testable, & maintainable requirements.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



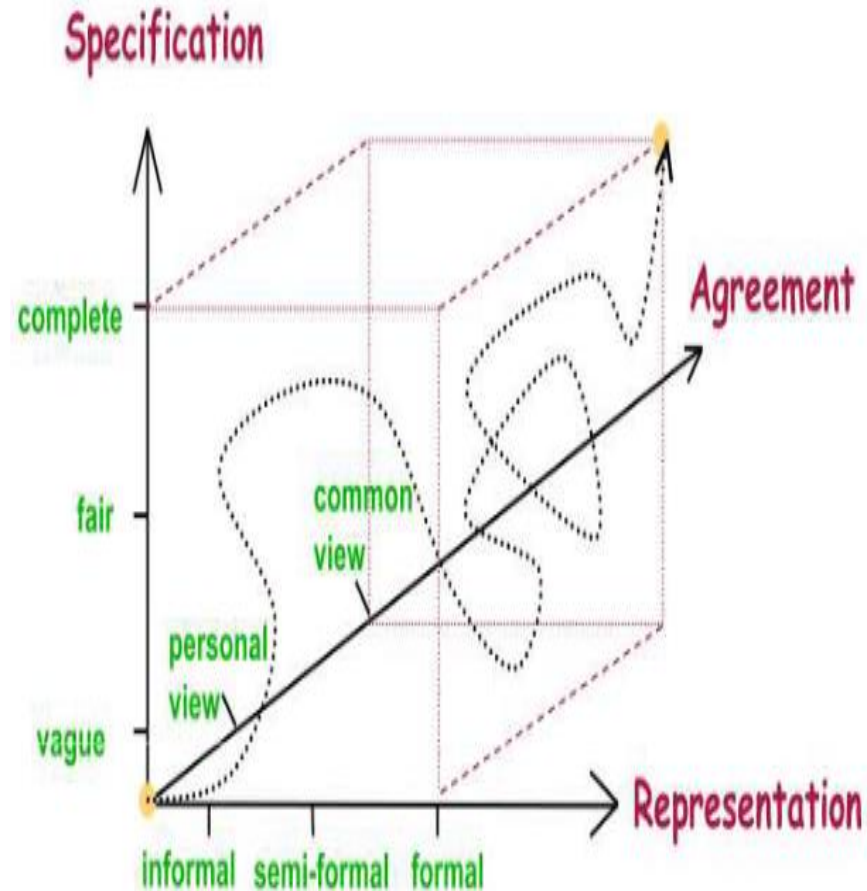
○ What is RE V and V?

- **Verification** aims to answer the question: “is **the system being built right?**”.
 - The process of determining that a **model implementation** and its **associated data** accurately represents the **developer’s conceptual description** and **specifications** [DOD, 2009].
- **Validation** aims to answer the question: “is the **right system** being built?”.
 - The process of determining the degree to which a **model or simulation** and its **associated data** are an accurate representation of the **real world** from the **perspective of the intended uses of the model** [DOD, 2009].

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

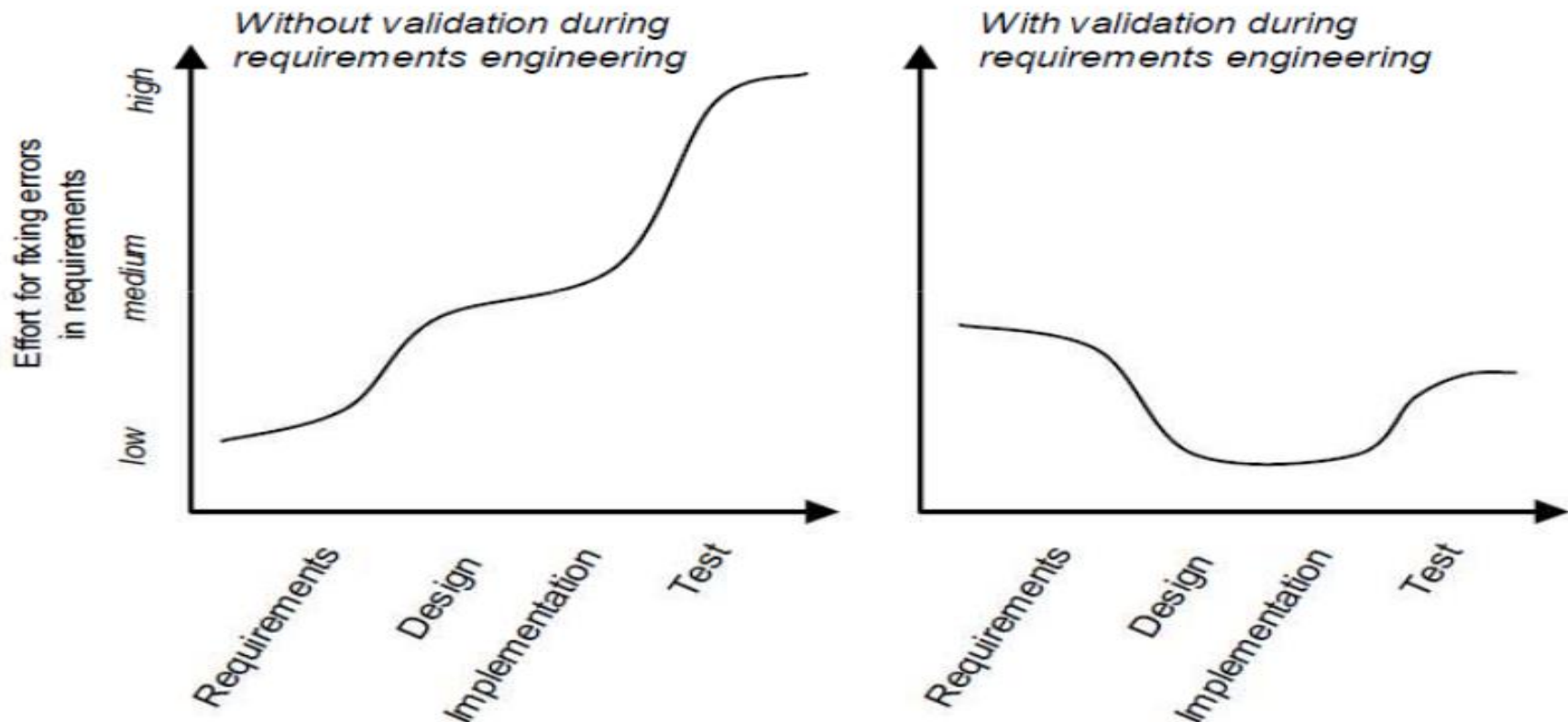
○ What is RE V and V?

- Have we **elicited**, **understood** and **specified** **ALL** requirements?
- Have we reached a **consensus** **among** **ALL** stakeholders?
- Are there still **conflicts** remaining?
- Have we specified **ALL** requirements in the **RIGHT** way? At the **good level** of details? With the **good precision**?



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **What is RE V and V ...?**
- **Cost of Requirements errors.**



Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



○ What is RE V and V ...?

- **Verification** and **validation** (V&V) are activities key to successful development.
 - While **verification** focuses on ensuring the program is built according to the **stated requirements**, the **verification** process includes activities such as **reviewing the code** and **doing walkthroughs** and **inspections**.
- The **verification process** ensures that software meets **specifications**.
- **Validation** focuses on whether the software meets the **expectations** and **requirements** of the end user.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Two Levels of Requirements V&V**

- **At the system level**

- Ensures that the software **being developed** (or **changed**) will **satisfy** its **stakeholders** (**Validation**).
 - Ensures that **each step** followed in the process of **building the software yields** the **right products** (**Verification**).



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Two Levels of Requirements V&V ...**

- **At the Requirements specification level**

- Checks the **software requirements specification** against stakeholders **goals** and **requirements** (**Validation**).
 - Checks **consistency** of the **software requirements specification artifacts** and other **software development products** (design, implementation, ...) against the **specification** (**Verification**).

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



○ When to perform Requirements V&V?

- Need to be performed **at every stage** during the **requirements** process.
- **Elicitation:** Checking back with the **elicitation sources**.
- **Analysis:** Checking that the **domain description** and **requirements** are **correct**.
- **Specification:** Checking **conformity** to **well-formed rules** and **standards**.
 - Checking that the defined **system requirement** will meet the **user requirements** under the **assumptions of the domain/environment**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



○ Principles to Requirements Validation

- The aim is **to ensure** that as **much errors** as possible are identified.
- There are **six** principles for **validation** of the Requirements.
 1. Involve with the **correct stakeholders**.
 2. Separate **errors identification** from their **correction**.
 - ✓ Independent V&V
 3. Validate requirements from **different perspectives**.
 - ✓ Reach a consensus.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Principles to Requirements Validation ...

4. Try **alternative formalisms**

- ✓ Use cases to understand **goals** and **interactions** with actors
- ✓ **State machines** for **transitions analysis**
- ✓ Class diagrams for **architecture** concerns
- ✓ May **highlight ambiguity**

5. Build **development items** from **requirements**

- ✓ Test cases, user manual, architecture diagrams.

6. **Repeat the validation** step (**continuous** validation)

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques

- Simple check
 - Traceability, well-written requirements.
- Prototyping
- Functional test design
- User manual development
- Reviews and inspections
- Model-Based V&V

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques ...

○ Simple check

- Various checks can be done using traceability techniques.
 - Given the **requirements document**, & verify that **all elicitation** notes are **covered**.
 - Tracing between different **levels of requirements**.
 - Checking **goals** against **tasks, features, & requirements**.
- Involves developing a **traceability matrix**
 - Ensures that the **requirements** have been taken into consideration (if not, there should be a reason).
 - Ensures that everything in the **specification** is **justified**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Simple check ...**

- Verify that the **requirements** are **well written** (according to the criteria already discussed).

- **Prototyping**

- Excellent for validation by users and customers.
 - More **accessible** than **specification**.
 - Demonstrate the **requirements** and help **stakeholders discover problems**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

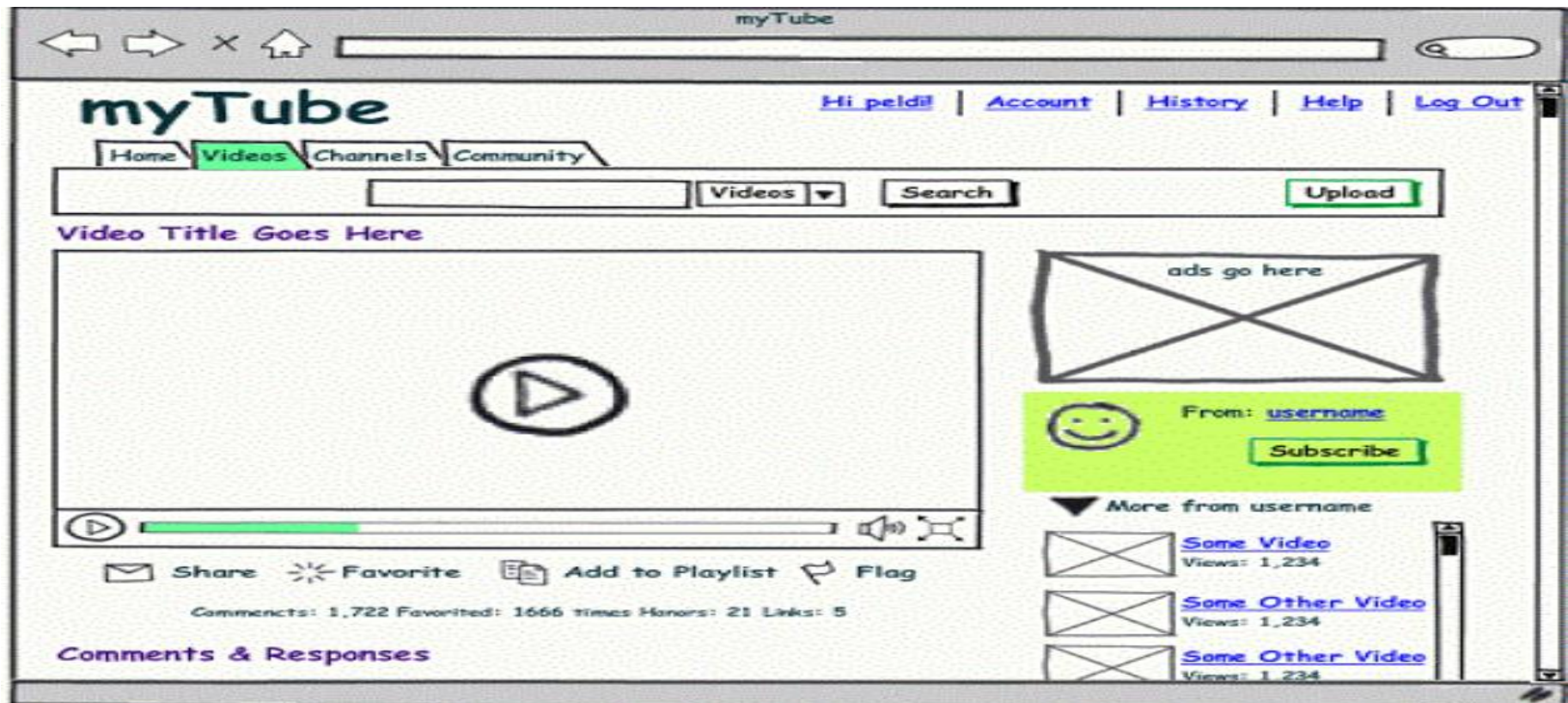


- **Requirements V&V Techniques ...**
 - **Prototyping ...**
 - Come in **all different shapes** and **sizes**
 - From paper **prototype of a computerized system** to **formal executable models/specifications** executable .
 - Horizontal, vertical.
 - Evolutive, **disposable** (more or less).



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

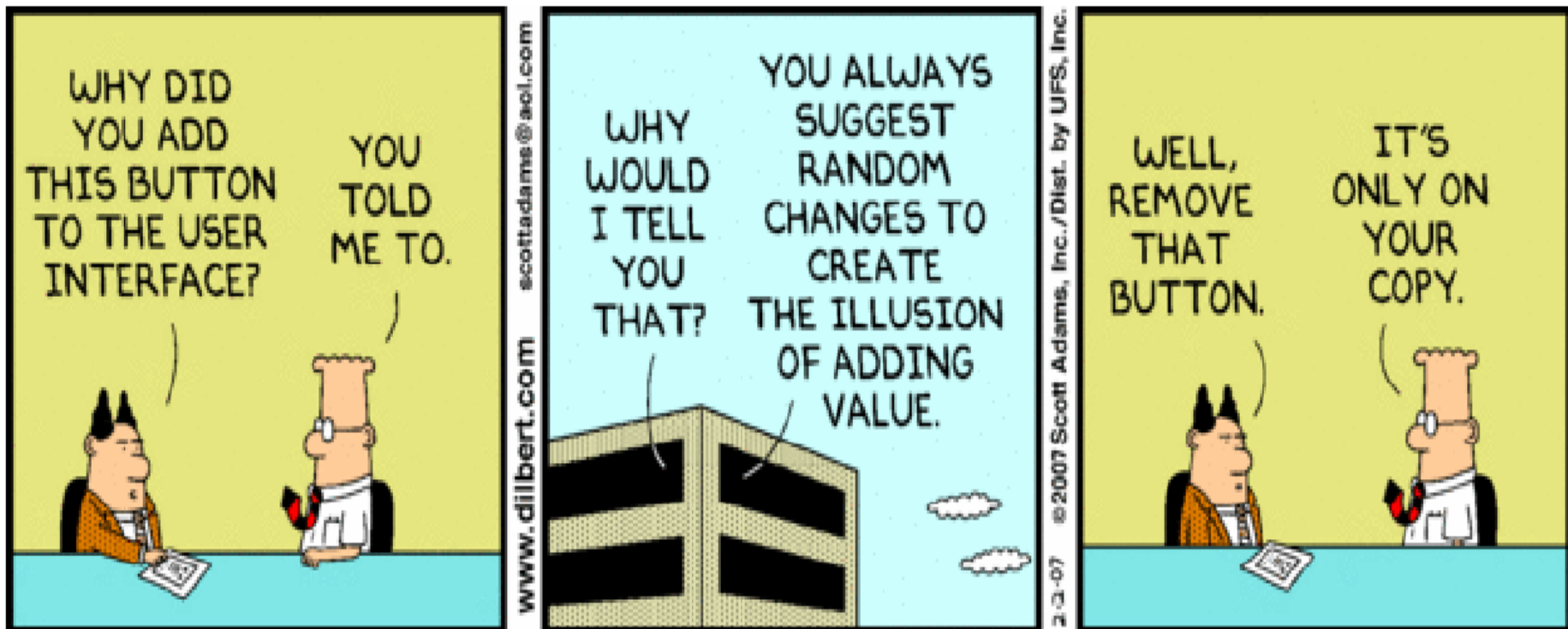
- **Requirements V&V Techniques ...**
 - **Prototyping ...** : the **prototype diagram** refers to the **different controls** such as **commands/tabs** and **captions** about **myTube**.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Prototyping ...**
 - Important to choose scenarios or use cases for elicitation.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques ...

○ Functional Test Design:

- Functional tests at the **system level** must be developed **sooner** or **later**.
 - Can (and should) be derived from the **requirements specification**.
 - Each (**functional**) requirement should have an **associated test**.
 - **Non-functional** (e.g., **reliability**) or **exclusive** (e.g., define what **should not happen**) requirements are **harder to validate with testing**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Functional Test Design ...**
 - **Each requirements test case** must be traced to **its requirements**.
 - **Inventing requirements** tests is an **effective validation technique**.
 - **Designing** these tests may **reveal errors** in the **specification** (even before **designing** and **building** the system)!
 - **Missing** or **ambiguous** information in the **requirements description** may make it **difficult** to **formulate tests**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Functional Test Design ...

- Some software development processes (e.g., **agile methods**) begin with **tests before programming** → **Test-Driven Development (TDD)**.





Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**
 - **Functional Test Design ...:** Java Example: **Production Code to verify if the result is as expected with Junit Annotation test.**

```
public class Student {  
    public String displayStudentName(String firstName, String lastName)  
    {  
        return firstName + lastName;  
    }  
}
```

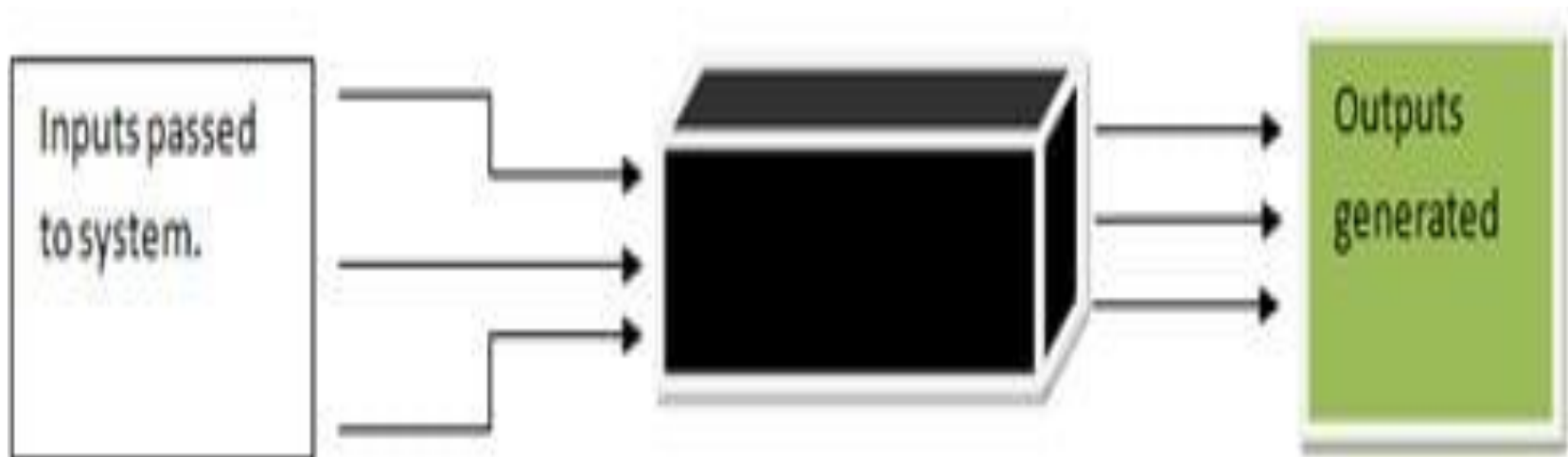
- **Test Code**

```
Import org.junit.Test;  
Import static org.junit.Assert.*;  
Public class StudentTest {  
    @Test  
    Public void testDisplayStudentName() {  
        Student student = new Student();  
        String studentName = student.displayStudentName("Anshuman", "Nain");  
        assertEquals("AnshumanNain", studentName);  
    }  
}
```

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Functional Test Design ...**
 - **Process Workflow for Functional Testing.**



Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **User Manual.**
 - Same reasoning as for functional test design.
 - Has to be done at some point.
 - Reveals problems earlier.
 - **Forces** a detailed look at requirements.
 - **Particularly useful** to describe **functional requirements** and **detect problems**.
 - **Particularly useful** if the **application** is **rich in user interfaces / for usability requirements**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **User Manual**
 - **Typical information** in a user manual.
 - Description of the **functionality**.
 - How to get **out of trouble**.
 - How to **install** and get **started** with the system.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Reviews and Inspections**
 - A *group of people* **read** and **analyze** requirements, **look for potential problems**, meet to discuss the problems, and agree on a list of action items needed to address these problem.
 - A **widely used requirements validation technique**.
 - Lots of evidence of effectiveness of the technique.
 - **Standardization: IEEE 1028 Standard for Software Review and Audits.**

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Reviews and Inspections ...**
 - Can be **expensive**
 - Careful **planning** and **preparation**.
 - **Pre-review** checking + **appropriate** checklists.
 - Different types of **reviews** with **varying degrees of formality**
 - Reading the document: A person **other than** the author of the document.
 - Reading and approval (sign-off): Encourages the reader to be more careful.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Reviews and Inspections ...**
 - Different types of reviews with varying degrees of formality ...
 - **Walkthroughs**
 - **Informal**, often **high-level overview**.
 - Can be led by **author/expert** to **educate others** on his/her work.
 - **Formal inspections**
 - Very **structured** and **detailed review**, defined roles for participants.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



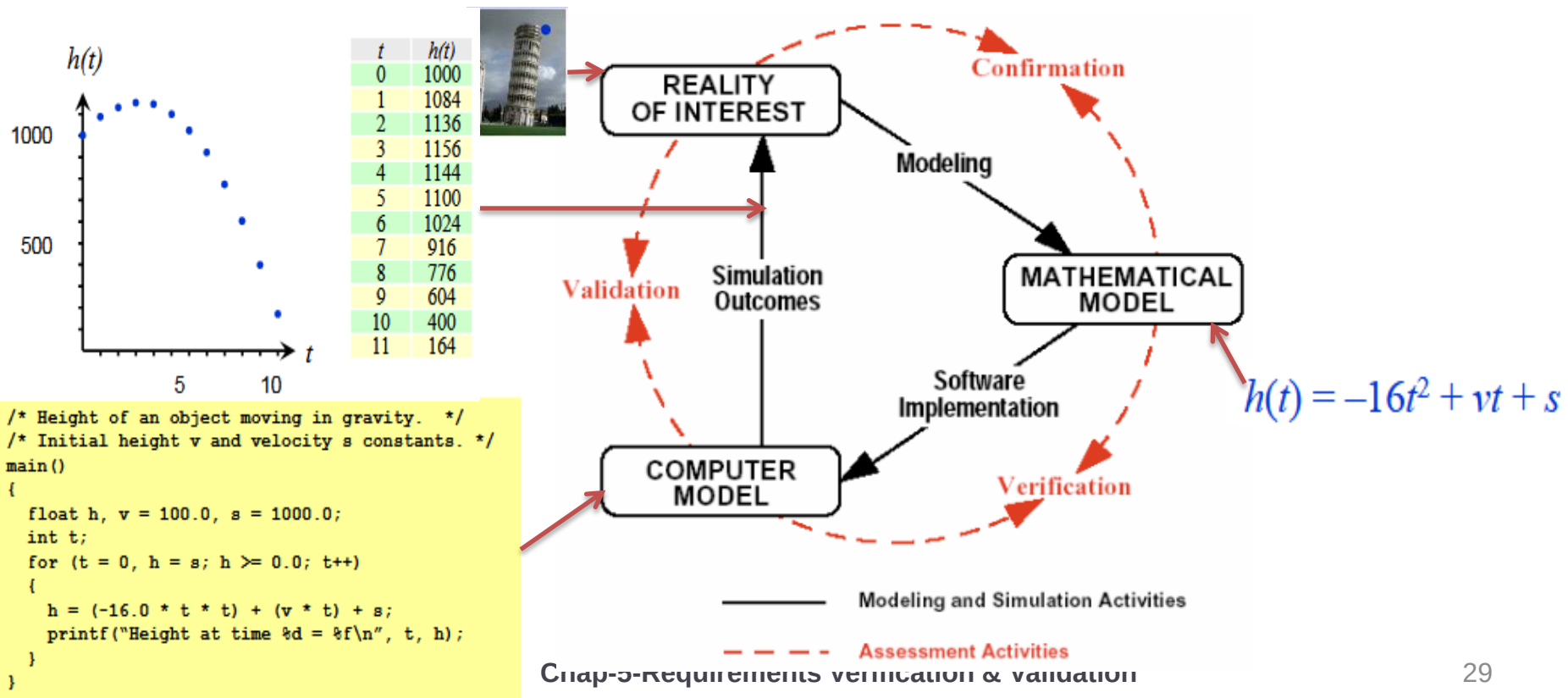
- **Requirements V&V Techniques ...**
 - **Reviews Checklists**
 - **Essential tool** for an effective review process.
 - List common problem areas and guide reviewers.
 - May include questions on **several quality aspects** of the document: **comprehensibility, redundancy, completeness, ambiguity, consistency, organization, standards compliance, traceability** .
 - **General checklists** and **checklists for particular modeling** and **specification languages** are important.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V

○ Simplified view of the model verification and validation



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V**
 - Concepts
 - **Model:** representation of something else.
 - **Simulation:** executing a model over time.

$$R = 2.59 \times 4 \sqrt{\sigma \times \left(\frac{\log^{-1}\left(\frac{ERP_t}{10}\right) \log^{-1}\left(\frac{G_r}{10}\right) \log^{-1}\left(\frac{MDS_r}{10}\right)}{\log^{-1}\left(\frac{FEL_r}{10}\right) F_t^2} \right)}$$

Model



Simulation

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

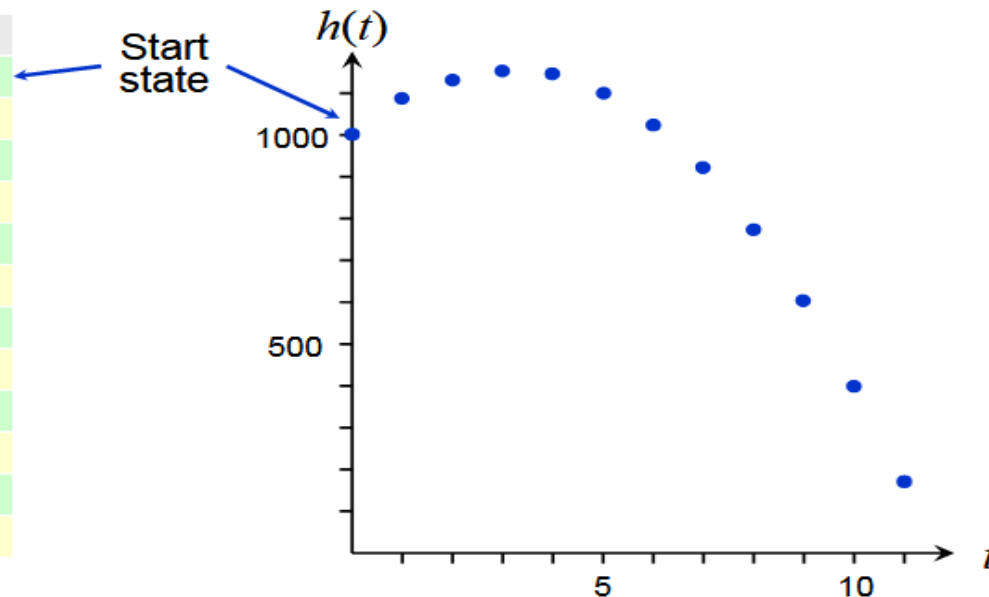
○ Model-based (Formal) V&V ...

○ Examples to **express Model**.

Example: Height under gravity

Model: $h(t) = -16t^2 + vt + s$ Data: $v = 100, s = 1000$

t	$h(t)$
0	1000
1	1084
2	1136
3	1156
4	1144
5	1100
6	1024
7	916
8	776
9	604
10	400
11	164

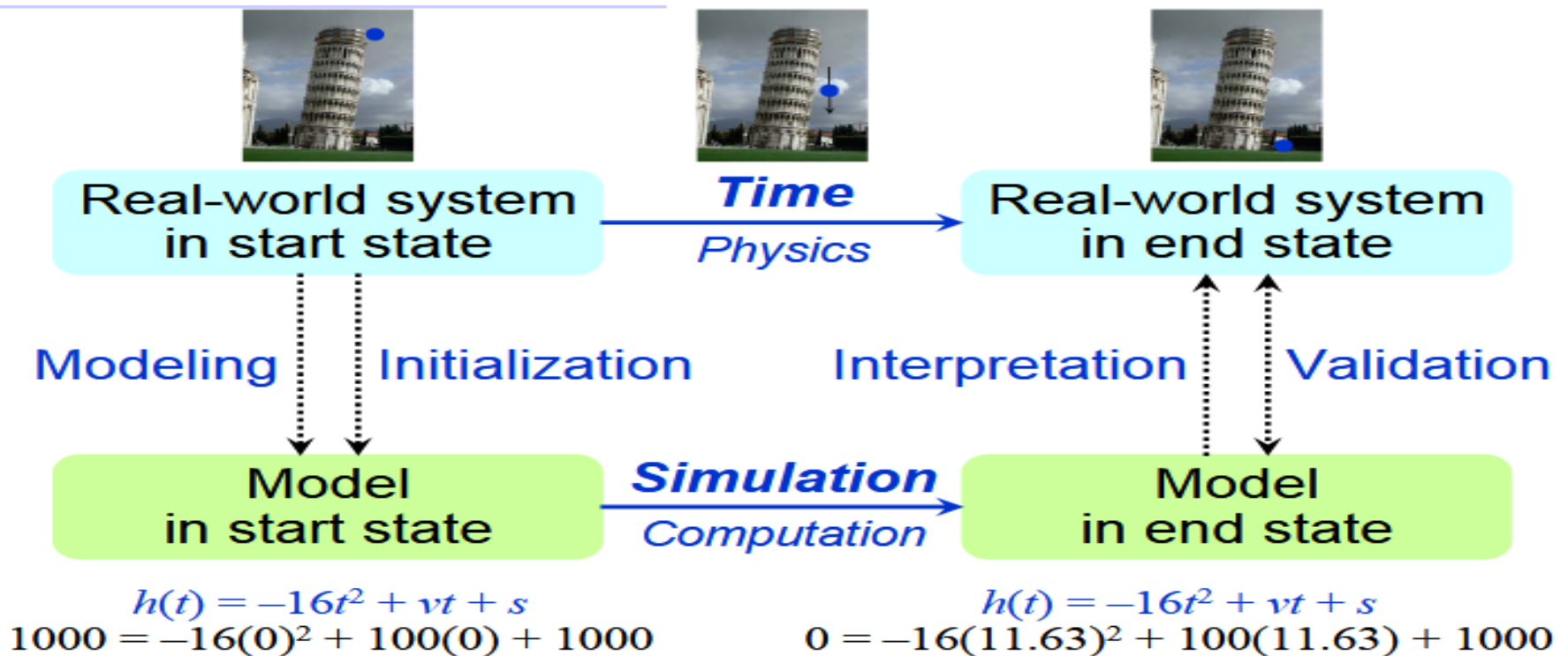


Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Simulation vs reality



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

- Relationship to express among the **Simulation, Model** and **Requirement** with Example.



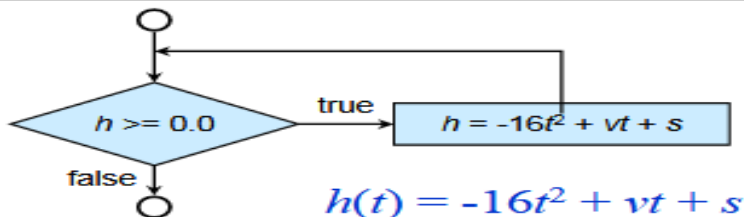
Simuland

- Real-world system
- Thing to be simulated



Requirements

- Intended uses
- Needed validity, resolution, scale



Conceptual model [Banks, 2010]

- Simuland components, structure
- Aspects of simuland to model
- Implementation specifications
- Use cases
- Assumptions
- Initial model parameter values

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

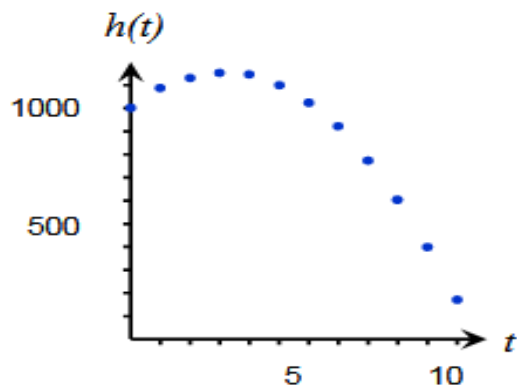
○ Model-based (Formal) V&V ...

○ Executable model and Results with Example.

```
/* Height of an object moving in gravity. */
/* Initial height v and velocity s constants. */
main()
{
    float h, v = 100.0, s = 1000.0;
    int t;
    for (t = 0, h = s; h >= 0.0; t++)
    {
        h = (-16.0 * t * t) + (v * t) + s;
        printf("Height at time %d = %f\n", t, h);
    }
}
```

Executable model

- Computer software
- Implemented conceptual model



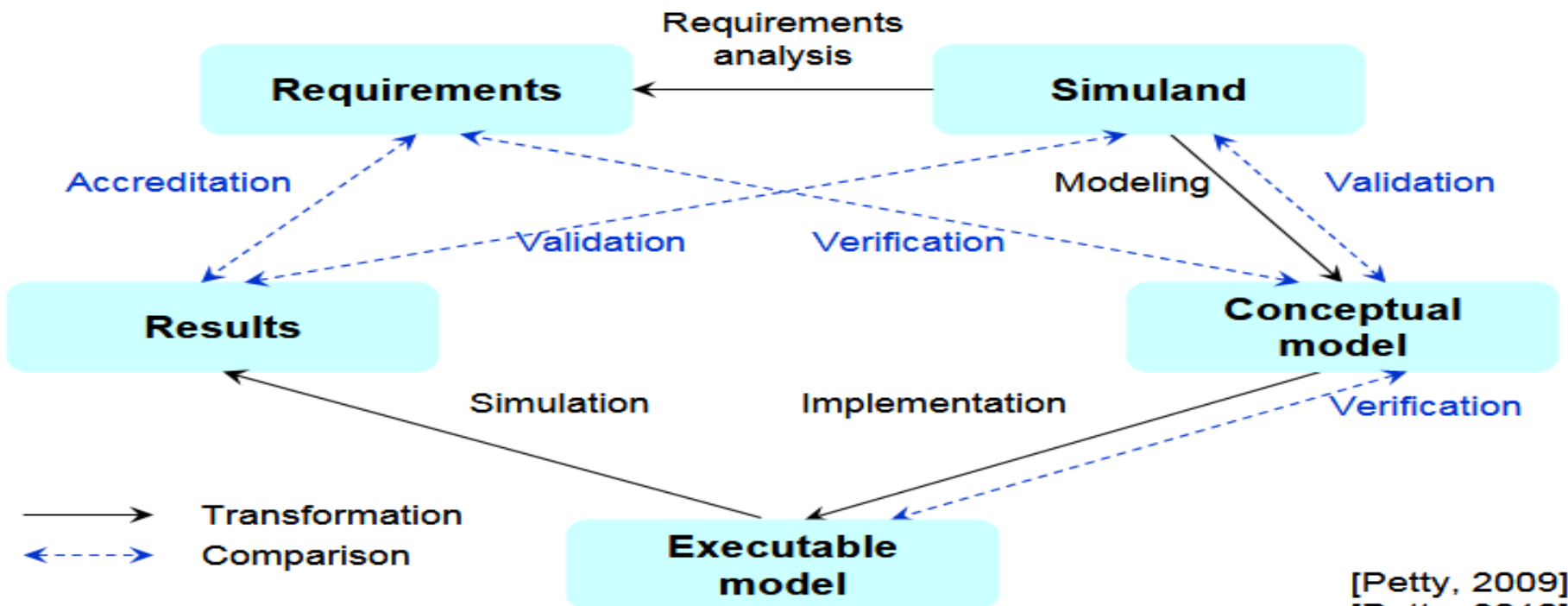
t	$h(t)$
0	1000
1	1084
2	1136
3	1156
4	1144
5	1100
6	1024
7	916
8	776
9	604
10	400
11	164

Results

- Output of model
- Produced during simulation

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - **VV&A comparisons**



[Petty, 2009]
[Petty, 2010]

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V**
 - **Types of Methods:** Modeling paradigms/Formal
 - Modeling paradigms
 - **Entity-Relationship modeling** – e.g. UML Class diagrams.
 - **Workflow modeling notations** – there are many different “dialects”, such as UML Activity diagrams, UCM, BPML.
 - **State machines** – e.g. Finite State Machines (FSM – a very simple formal model), extended FSMs, such as UML State diagrams model), extended FSMs, such as UML State diagrams.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **First-order logic** – notations such as Z, UML-OCL, etc.
 - Can be used as **an add-on** with the **other paradigms** above, by providing information about **data objects** and **relationships** (possibly in the form of “**assertions**” or “**invariants**” that hold at certain points during the **dynamic execution** of the model).

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **First-order logic ...**
 - Can be used alone, expressing **structural models** and **behavioral models** (there are many examples of using Z for such purpose).

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **First-order logic** –....
 - Example: UML standardized by the OMG (Object Management Group).
 - Like previous **object-oriented notations**, UML **does not enable** to express **all integrity constraints**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **First-order logic –....**
 - Some **complementary proposals** have been done to better **integrate constraint expressions** in UML.
The proposal suggests **two options** for handling the **constraints**.
 - One is to support **extension of meta-model adding** to **each class a part** where **constraints are written**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **First-order logic ...**
 - The proposal
 - Two is to apply the **Object Constraint Language (OCL)** adds **constraints expressed in first order logic** to **the diagrams**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **OCL language** makes possible to **express constraints precisely** and **without ambiguity**.
 - **Unfortunately**, the current OCL tools **do not exploit this preciseness**.
 - Therefore, the **Formal languages** have **precise notations** based on **mathematical concepts** which enable to do **proofs** on **specifications**. The typical Tool is **RoZ**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **RoZ**, is integrated in the **Rational Rose environment**.
 - It allows the **integration of data specifications expressed in UML** with **formal annotations** in the **Z language**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - Starting from an **annotated specification**, the tool automatically produces a **formal specification** by **translating the UML constructs** and **merging** them with the **annotations expressing predicates not included** in the graphical design.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **RoZ** can also complete the **UML class diagram** by **automatically generating** in the UML **environment Z specifications of elementary operations** on the classes (attributes **modifications**, **adding** and **deleting** objects of the class).

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - **RoZ Principle:** to create **formal specification** from UML.
 - **RoZ** is an extension of the **Rational Rose** tool to integrate **UML** and **Z notations**.
 - The **Rose environment** is used to build the **UML models** which are completed by **formal annotations** in **Z**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - ***RoZ Principle ...***
 - The **annotations** are expressed into **forms**.
 - A form should contain **all the information mandatory** to complete **Z skeletons**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - **Methods**
 - Formal approaches ...
 - From a **class diagram** and the **information contained** in its forms, **RoZ** uses **Rose scripts** to :
 - **generate Z specifications**
 - From a UML class diagram and its annotations, a complete Z specification can be generated in a file.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - **Methods**
 - Formal approaches ...
 - From a **class diagram** and the **information contained** in its forms, **RoZ** uses Rose scripts to :
 - **generate elementary operations**
 - The main idea is that for some concepts, there are operations which always exist.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - From a **class diagram** and the **information contained** in its forms, **RoZ** uses Rose scripts to
 - **generate elementary operations ...**
 - For instance, a class has **always operations** to **modify its attributes**.
 - Moreover if the **class** is **concrete**, there are also operations **to add** or **remove** an instance to or from the **class instances**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- From a **class diagram** and the **information contained** in its forms, **RoZ** uses Rose scripts to

- **generate theorems to validate operation guards**

- From a UML class diagram, **elementary operations** and a **Z formal specification** can be generated **automatically**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- From a **class diagram** and the **information contained** in its forms, **RoZ** uses Rose scripts to

- **generate theorems to validate operation guards ...**

- Data must **often respect constraints** which **are not taken into account** by the **code generated automatically** and **only implicitly** taken into account in **the specification**.



Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ

- Assume the descriptions of a simplified version of the **access control system model** presented. This model features **two classes**: **“PERSON”** and **“GROUP”**. Each **person** has **four attributes**: **his last and first names**, the set of **his telephone numbers** and the **number of his magnetic card**. Each **group** is characterized by a **name** (e.g. **“Accounting Department”**) and a **code** (e.g. **“AccD”**). **One** can note that the **types of the attributes** are expressed in **the Z style**. For instance, the **“tel”** attribute is **multi-valuated** i.e. it corresponds to a set of **telephone numbers (\finsettel)**.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

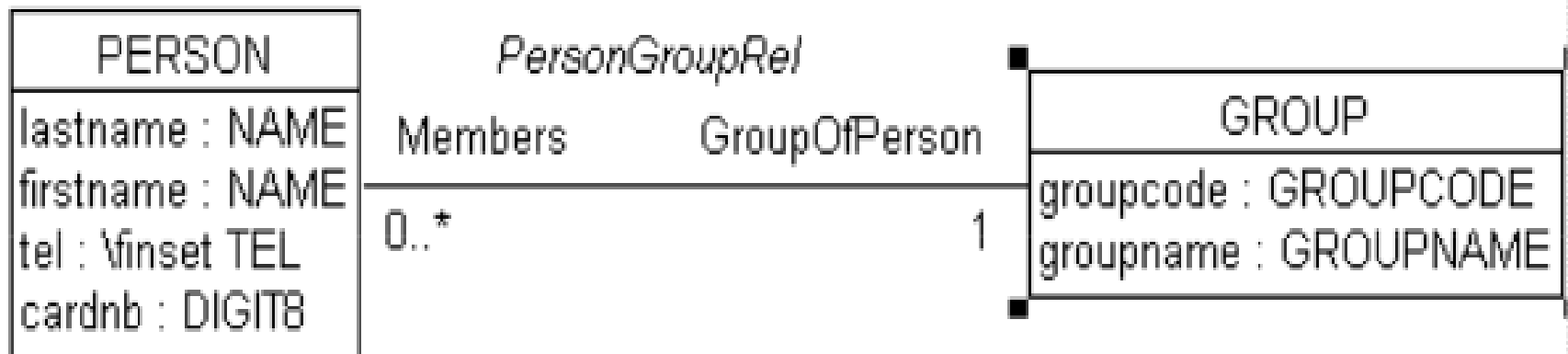
○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Formal approaches ...

○ Construction of a Class Diagram Example in RoZ ...

- The **PersonGroupRel** association links **each person** to **one group**. The figure below taken from Rose interface - the access control class diagram.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Formal approaches ...

○ Construction of a Class Diagram Example in RoZ ...

- This class diagram in **slide 52** does not express the full specification of the application data.

- **Three** additional **constraints complement this diagram:**

1. Every person has at least **one telephone number**.
2. The card number is **a key of persons**.
3. The telephone numbers of the members of a given group have the **same prefix**.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - Construction of a Class Diagram Example in RoZ ...
 - The UML class diagram of the **access control system** is built in a **class diagram** window of the **Rose environment**.
 - The first constraint (**every person** has at **least one telephone number**) holds on the attribute “**tel**”.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - Construction of a Class Diagram Example in RoZ ...
 - So it is written in the **field “Documentation”** of the attribute **“tel”** in the diagram.
 - It is expressed in Z that the **set of telephone numbers cannot be empty :**

***tel* $\neq \emptyset$**

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- Requirements V&V Techniques ...

- Model-based (Formal) V&V ...

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ ...

- The second constraint (the **card number** is a key of persons) means that **two different persons** have **distinct card numbers**.

- It is a comparison between existing **objects of the class**, so it is in the field documentation of **the class "PERSON"**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Formal approaches ...

○ Construction of a Class Diagram Example in RoZ ...

- The second constraint (the card number is a key of persons) means that two different persons have distinct card numbers.
- It is a comparison between existing objects of the class, so it is in the field documentation of the class “PERSON”.

$\forall p1, p2 : \text{Person} \mid p1 \neq p2 \bullet p1.\text{cardnb} \neq p2.\text{cardnb}$



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ ...

The image displays two side-by-side windows from the RoZ software, illustrating the construction of a class diagram.

Left Window: Class Attribute Specification for tel

- General** tab is selected.
- Name:** tel
- Class:** PERSON
- Type:** Minset TCL
- Initial value:** (empty)
- Export control:** Public, Protected, Private, Implementation (all unchecked).
- Documentation:** tel \neq \empty

Right Window: Class Specification for PERSON

- General** tab is selected.
- Name:** PERSON
- Parent:** Logical View
- Type:** Class
- Stereotype:** (empty)
- Export control:** Public, Protected, Private, Implementation (all unchecked).
- Documentation:** Morall p1,p2 : Person | p1 \neq p2 @
p1.cardnb \neq p2.cardnb

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ ...

- To express the **third constraint** (the telephone numbers of the members of **a given group have the same prefix.**).

- This constraint is related to the relationship between **“PERSON”** and **“GROUP”**, so it is expressed in the **“Documentation”** field of **“PersonGroupRel”**.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Formal approaches ...

○ Construction of a Class Diagram Example in RoZ ...

- Let us consider that we have defined a “**prefix**” function which gives the **prefix** of a **telephone number**.
- The constraint states that for **two persons** which belong to the **same group**, **each telephone number** of their sets of **telephone numbers** have **the same prefix**. :

$\forall p1, p2 : \text{Person} \mid$

$\text{GroupOfPerson}(p1) = \text{GroupOfPerson}(p2) \bullet$

$\forall t1 : p1.\text{tel} \bullet \forall t2 : p2.\text{tel} \bullet$

$\text{prefix}(t1) = \text{prefix}(t2)$

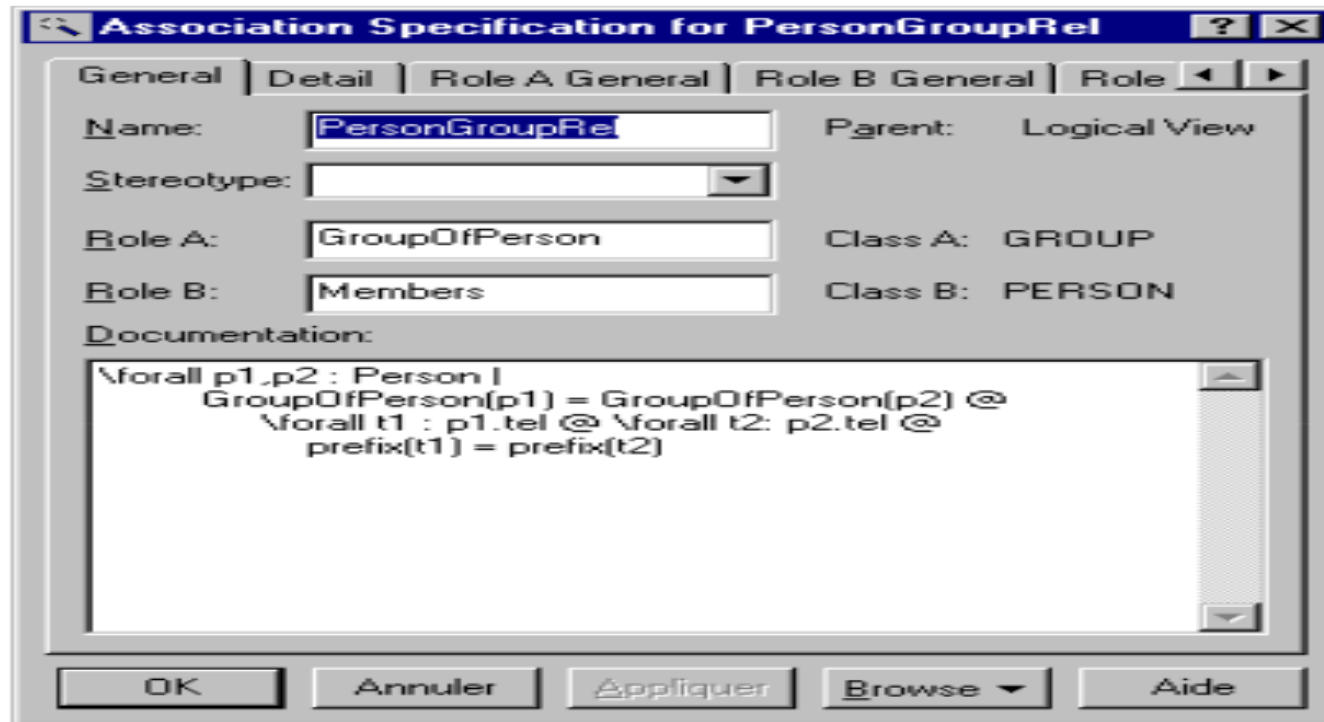
Week 9, 10 and 11 Lessons: Requirements Verification & Validation

- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ ...



Association Specification for PersonGroupRel

General | Detail | Role A General | Role B General | Role

Name: Parent: Logical View

Stereotype:

Role A: Class A: GROUP

Role B: Class B: PERSON

Documentation:

```
\forall forall p1,p2 : Person |
  GroupOfPerson(p1) = GroupOfPerson(p2) @
  \forall forall t1 : p1.tel @ \forall forall t2: p2.tel @
    prefix[t1] = prefix[t2]
```

OK Annuler Appliquer Browse Aide

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - Formal approaches ...
 - Construction of a Class Diagram Example in RoZ ...
 - **Generation of a Z Specification ...**
 - RoZ automatically generates **the Z schema skeletons** corresponding to a UML class diagram.
 - The **Z skeletons** are produced by translation of the concepts (**class, association, inheritance...**) of the diagram.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- Formal approaches ...

- Construction of a Class Diagram Example in RoZ ...

- **Generation of a Z Specification ...**

- The constraint on the telephone numbers (constraint 1) is expressed in the Z schema defining attributes,
 - while the key constraint (constraint 2) is defined in the Z schema representing the existing objects of **“PERSON”**. (See next diagram).



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirements V&V Techniques ...

○ Model-based (Formal) V&V ...

○ Formal approaches ...

○ Construction of a Class Diagram Example in RoZ ...

○ Generation of a Z Specification ...

PERSON

lastname : *NAME*

firstname : *NAME*

tel : $\mathbb{F} TEL$

cardnb : *DIGIT8*

tel $\neq \emptyset$

PersonExt

Person : $\mathbb{F} PERSON$

$\forall p1, p2 : Person \mid p1 \neq p2 \bullet p1.cardnb \neq p2.cardnb$

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**
 - **Model-based (Formal) V&V ...**
 - **Operations on Models**
 - **Available V&V techniques** will **vary** from **one modeling paradigms to another** and will also depend on the available tools (that usually only apply to a particular “dialect” of the modeling paradigm).

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- **Operations on Models ...**

- The following functions may be provided through tools:

- **Completeness checking** – only according to certain syntax rules, templates.

- **Consistency checking: given model M**, show that M **does not** imply a contradiction and **does not** have any other undesirable general property (e.g. deadlock possibility)

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirements V&V Techniques ...**

- **Model-based (Formal) V&V ...**

- **Operations on Models ...**

- The following functions may be provided through tools:

- **Refinement checking:** given **two models M** and **M'**, show that the **properties of M** imply the properties of **M'**.

- **Model checking:** given a **model M** and some properties **P**, show that any system implementation **satisfying M** will have **the properties P**.

- **Theorem proving:** prove (through induction or other approach) the **correctness** and **validity of theorems** against a **model**, for **all situations**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



○ Requirement Prioritization:

- **Software requirement prioritization** is a process in software engineering that precludes the **actual development of software application programs** and deals with **assigning priorities** to **single requirements** to define the **order of their implementation**.
- **Requirement prioritization** is used in **software product management** for determining which **candidate requirements** of a software product should be included in a **certain release**.
- **Requirements** are also prioritized to minimize risk during development so that the **most important or high risk requirements** are **implemented first**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



○ Requirement Prioritization:

- A **good** and **relatively easy** to use **method** for **prioritizing software product requirements** is the **cost-value approach**.
- The basic idea of **cost-value approach** is to determine for each individual candidate requirement what the cost of implementing the requirement shall be and how much value the requirement has.
- **Analytic Hierarchy Process** (AHP) is used to perform the assessment of **values** and **costs** for the requirements in **cost-value approach**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization: AHP Method**

- **Analytic Hierarchy (AHP) process**

- AHP is a method for decision making in situations where **multiple objectives** are present.
 - This method uses a **pair-wise comparison matrix** to calculate the relative **value** and **costs** of security requirements.
 - By using AHP, the requirements engineer can also confirm the **consistency of the result**.
 - **AHP** can prevent **subjective judgment errors** and **increase** the likelihood that the results are **reliable**.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Analytic Hierarchy (AHP) process ...**

- There are five steps in the AHP method:
 - Review candidate requirements for completeness.
 - Apply the pair-wise comparison method to assess the relative value of the candidate requirements.
 - Apply the pair-wise comparison method to assess the relative cost of implementing each candidate requirement.
 - Calculate each candidate requirement's relative value and implementation cost, and plot each on a cost-value diagram.
 - Use the cost-value diagram as a map for analyzing the candidate requirements.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirement Prioritization ...:

○ Analytic Hierarchy (AHP) process ...

○ Prioritization Method Comparison

- The candidate prioritization methods are to be compared so that a suitable method can be selected.
- Some **example evaluation criteria** are:
 - **clear-cut steps:**
 - There is clear definition between stages or steps within the prioritization method.
 - **quantitative measurement:**
 - The prioritization method's numerical output clearly displays the clients' priorities for all requirements.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirement Prioritization ...:

○ Prioritization Method Comparison ...

○ Some example evaluation criteria are ...:

○ **high maturity:**

○ The method has had considerable exposure and analysis in the requirements engineering community.

○ **low labor-intensity:**

○ A reasonable number of hours are needed to properly execute the prioritization method.

○ **shallow learning curve:**

○ The requirements engineers and stakeholders can fully comprehend the method within a reasonable length of time.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirement Prioritization ...:

- **Typical Examples-Reading Article:** Mead, N. R. (2008). Requirements prioritization case study using AHP. *Software Engineering Institute*.
- **Analytic Hierarchy (AHP) process ...**
 - Prioritization of software requirements is a sub process of the release planning process.
 - The release planning process consists of the sub processes:
 - Prioritize requirements
 - Select requirements
 - Define release requirements
 - Validate release requirements
 - Prepare launch

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality**

- **Quality Function Deployment (QFD)** is process or set of tools used to define the customer requirements for product and convert those requirements into engineering specifications and plans such that the customer requirements for that product are satisfied.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- QFD aims at translating Voice of Customer into measurable and detailed design targets and then drives them from the assembly level down through sub-assembly level, component level, and production process levels.
 - QFD helps to achieve structured planning of product by enabling development team to clearly specify customer needs and expectations of product and then evaluate each part of product systematically.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirement Prioritization ...:

○ Quality Function Deployment (QFD) in Software Quality ...

○ Key steps in QFD :

○ Product planning

- Translating what customer wants or needs into set of prioritized design requirements.
- Prioritized design requirements describe looks/design of product.
- Involves benchmarking – comparing product's performance with competitor's products.
- Setting targets for improvements and for achieving competitive edge.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Key steps in QFD :**

- **Part Planning**

- Translating product requirement specifications into part of characteristics.
 - For example, if requirement is that product should be portable, then characteristics could be light-weight, small size, compact, etc.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Key steps in QFD :**

- **Process Planning**

- Translating part characteristics into an effective and efficient process.

- **Production Planning**

- Translating process into manufacturing or service delivery methods.
 - In this step too, ability to deliver six sigma quality should be improved.

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Example: A QFD Tool – House Of Quality (HOQ) :**

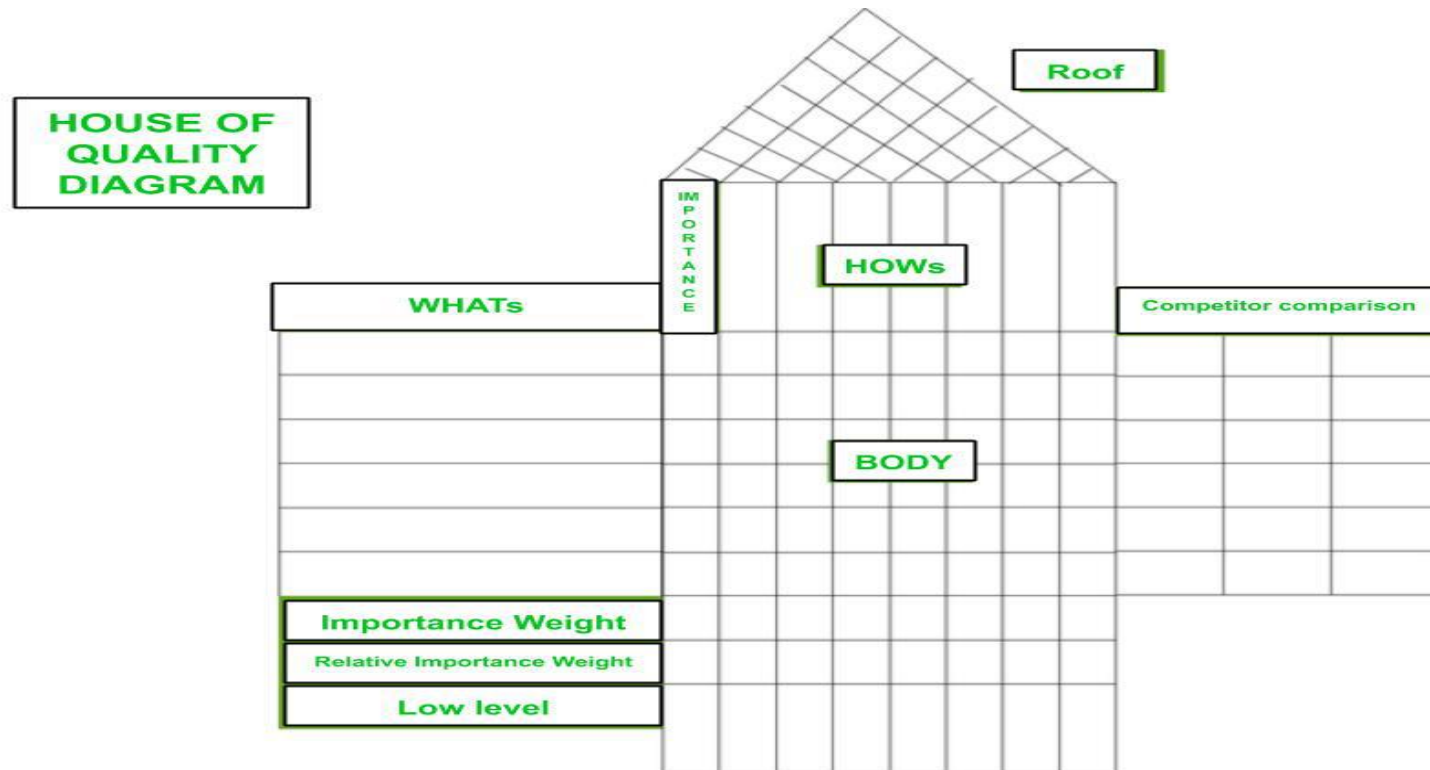
- **Descriptions:** House of Quality or HOQ is conceptual map or matrix that provides an understanding of how customer requirements (WHATs) are related to various technical descriptors or design parameters (HOWs) and their priority levels. House of Quality is also known as **Quality Matrix**. The matrix gets its name from fact that it represents the shape of house.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirement Prioritization ...:

○ Quality Function Deployment (QFD) in Software Quality ...

○ **Example: Diagram – House Of Quality (HOQ) :**



Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Core Features of HOQ**

- **WHATs** - Customer requirements and needs are listed.

- **HOWs**

- It comprises design features, technical descriptors and specifications of product aligned with customer requirements.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Core Features of HOQ ...**

- **Importance Factor**

- The team rates each of customer requirements (WHATs) on scale of 1 to 5 based on their level of importance to the customer. Here, 1 denotes lowest level and 5 denotes highest level of importance to customer.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



○ Requirement Prioritization ...:

○ Quality Function Deployment (QFD) in Software Quality ...

○ Core Features of HOQ ...

○ Body

- HOWs are ranked on basis of their correlation of satisfying each of listed WHATs.
- Body Ranking System used is set of symbols used to show **Strong, Moderate, Weak** or **No correlation** between **HOWs** and **WHATs**.
- Also, each of symbols represents numerical value.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Core Features of HOQ ...**

- **Roof**

- The roof indicates how design requirements(HOWs) are related to each other. Roof Ranking System uses set of symbols to represent different types of interactions – **Strong Positive, Positive, None, Negative or Strong Negative.**

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**
 - **Quality Function Deployment (QFD) in Software Quality ...**
 - **Core Features of HOQ ...**
 - **Competitor Comparison**
 - This part focuses on comparing competitor's product in regards to fulfilling WHATs. This is also measured on scale of 1 to 5 where 1 denotes Highly Dissatisfied and 5 denotes Highly Satisfied.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- **Core Features of HOQ ...**

- **Relative Importance**

- This part gives results by calculating total sum of each of HOWs by multiplying their Importance with value of Body Ranking symbol.
 - This part is useful as it allows us to identify HOWs of products which require more attention and resources.

Week 9, 10 and 11 Lessons: Requirements Verification & Validation



- **Requirement Prioritization ...:**
 - **Quality Function Deployment (QFD) in Software Quality ...**
 - **Core Features of HOQ ...**
 - **Lower Level or Foundation**
 - This part of HOQ lists more specific target values for technical specifications in relation to HOWs in order to satisfy customer requirements.



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirement Prioritization ...:

- Quality Function Deployment (QFD) in Software Quality ...
 - The **Roof Ranking System** and **Body Ranking System** are as follows :

ROOF RANKING SYSTEM	
++	Strong Positive
+	Positive
	None
-	Negative
--	Strong Negative

BODY RANKING SYSTEM		
●	Strong	9
○	Moderate	3
△	Weak	1
	No Correlation	0

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- The formula for Computing Importance Weight and Relative Importance Weight :

- Importance Weight, = Sum of (Importance Factor * Body Ranking Symbol Value) in column.
 - Relative Importance Weight, = (Importance Weight/ Total Importance Weight) * 100



Week 9, 10 and 11 Lessons: Requirements Verification & Validation

○ Requirement Prioritization ...:

○ Quality Function Deployment (QFD) in Software Quality ...

- The formula for Computing Importance Weight and Relative Importance Weight :

Importance Factor	Technical Requirement A
2	⊙
3	○
3	△
1	
4	
5	

Week 9, 10 and 11 Lessons:

Requirements Verification & Validation



- **Requirement Prioritization ...:**

- **Quality Function Deployment (QFD) in Software Quality ...**

- The formula for Computing Importance Weight and Relative Importance Weight :

$$\text{Importance Weight (A)} = (2 * 9 + 3 * 3 + 3 * 1) = 18 + 9 + 3 = 30$$

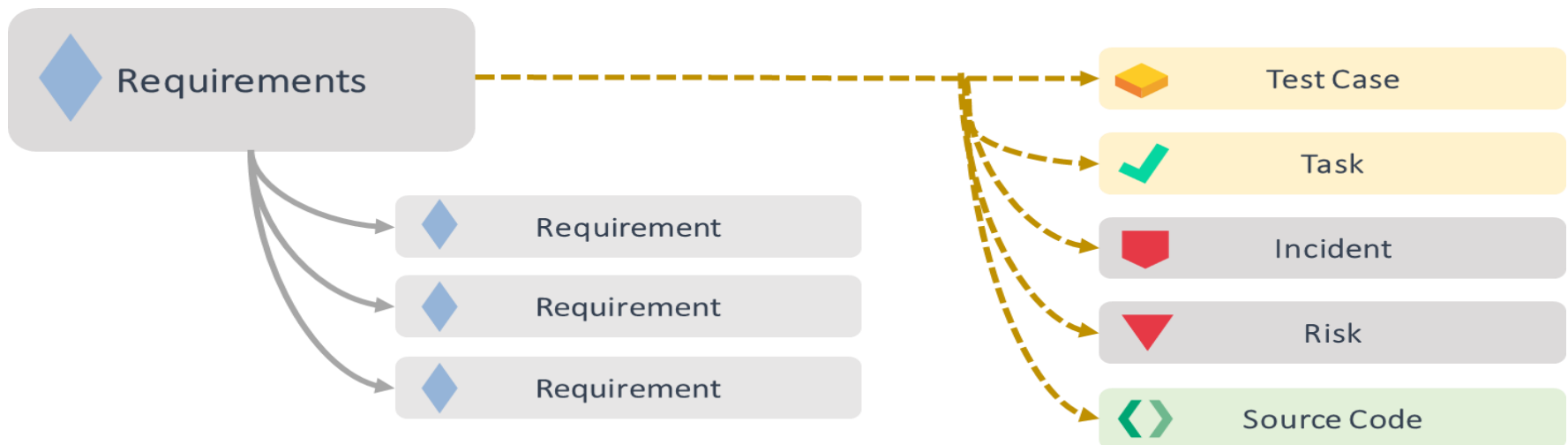


Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

○ Requirements Traceability:

- It refers to the ability to describe and follow the life of a requirement, in both **forwards** and **backwards** direction (i.e. from its **origins**, through its **development** and **specification**, to its subsequent **deployment** and **use**, and through all periods of on-going refinement and iteration in any of these phases.)
- It is the ability to trace a requirement forwards and backwards in the development lifecycle.





Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

○ Requirements Traceability ...:

- Requirements are traced forward through other development artifacts, including test cases, test runs, and issues.
- Requirements are traced backward to the source of the requirement, such as a stakeholder or a regulatory compliance mandate.
- The purpose of requirements traceability is to verify that requirements are met.
 - It also accelerates development.
 - That's because it's easier to get visibility over your requirements.
 - Traceability is also important for analysis.
 - If a requirement changes, then you can use traceability to determine the impact of change.



Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

○ Requirements Traceability ...

- The activity also ensures that all of the requirements are internally consistent with each other and support the overarching business drivers, goals and objectives.

○ Requirements Traceability Matrix

- The most common way of ensuring that there is full requirements traceability is by means of a **Requirements Traceability Matrix (RTM)**.
- The traceability matrix is used to verify that all stated and derived requirements are associated with corresponding design elements, system components, modules and other project deliverables. This is known as the **forward trace**.
- The RTM is also used to verify and document the original source of the requirements so that if questions should arise by the customer regarding why certain features are included, there is a comprehensive audit trail. This is known as the **backward trace**.



Week 9, 10 and 11 Lessons:

Requirements Verification & Validation

○ Requirements Traceability ...

○ Requirements Traceability Matrix (RTM) Tools

- **Visure Requirements:** This tool is provided by Visure Solutions, which is focused on business-critical and safety-critical industries. Its Visure Requirements tool provides complete traceability.
- **Modern Requirements4DevOPs:** This tool is integrated with Microsoft's Azure DevOps, TFS, and VSTS, and gives project managers traceability through every stage of the process.
- **ReQtest:** Providing traceability from project start to finish; this tool is based in the cloud. It has a very customizable requirement module that assists project managers in quickly evaluating and tracing changes.